

Assignment & Presentation (30%)

English-French Machine Translation with Encoder-Decoder LSTM Model

1. Regulations

- **Project group** : Maximum **3 students** .
 - **Deadline** : **November 22, 2025** (23:59).
 - **Submission form** : **01 single PDF file** (report + source code in appendix) via **E-Learning system** .
 - **No late submissions accepted** .
-

2. Introduction

The exercise requires students **to implement from scratch an Encoder-Decoder** model using **LSTM** to solve the problem of **machine translation from English to French** . The model uses **a fixed context vector** from the Encoder (not required to use the attention mechanism).

Students will:

- Parallel string data processing.
- Train the model with PyTorch.
- Perform performance forecasting and evaluation.
- Analyze errors and suggest improvements.

Tools : Python + PyTorch (do not use available seq2seq libraries like `torchtext.legacy` or `transformers`).

3. Objectives

1. **Encoder-Decoder LSTM** architecture with **fixed context vector** .
 2. Process string data, train, evaluate by **BLEU score** .
 3. Analyze translation errors and suggest improvements (e.g. add attention, beam search...).
-

6. Implementation instructions

6.1. Data preparation

Setting

```
pip install spacy torch torchtext
```

```
python -m spacy download en_core_web_sm
```

```
python -m spacy download fr_core_news_sm
```

Tokenization

```
from torchtext.data.utils import get_tokenizer
```

```
en_tokenizer = get_tokenizer('spacy', language='en_core_web_sm')
```

```
fr_tokenizer = get_tokenizer('spacy', language='fr_core_news_sm')
```

Building a dictionary (Vocabulary)

- Use `build_vocab_from_iterator` .
- Add special tokens: `<unk>` , `<pad>` , `<sos>` , `<eos>` .
- Limit: **10,000 most common words** per language.

Padding & Packing

- Use `pad_sequence` to synchronize length in batch.
- Use `pack_padded_sequence` before entering LSTM.
- **Sort batch by decreasing length** → `enforce_sorted=True` .

DataLoader

- Batch size: **32–128**
- Use custom `collate_fn` to handle padding and packing.

6.2. Model building

Encoder

$$(h_t, c_t) = \text{LSTM}(\text{embed}(x_t), (h_{t-1}, c_{t-1}))$$

- Input: English token string → embedding (size 256–512).
- Output:
 - Hidden states chain $(h_1, \dots, h_n)$
 - **Context vector** $= (h_n, c_n) \rightarrow$ passed to Decoder.

Decoder

$$(h_t, c_t) = \text{LSTM}(\text{embed}(y_{t-1}), (h_{t-1}, c_{t-1}))$$

$$p(y_t) = \text{softmax}(\text{Linear}(h_t))$$

- Input:
 - Initial: <eos> + context vector from Encoder.
 - Next steps: previous token (ground truth or prediction).
- Output: Probability of next word in French.

Recommended parameters

Parameters	Value
Hidden size	512
Embedding dim	256–512
Number of LSTM layers	2
Dropout	0.3–0.5
Teacher forcing ratio	0.5

6.3. Model training

Ingredient	Configuration
Loss	<code>nn.CrossEntropyLoss(ignore_index=pad_idx)</code>
Optimizer	<code>Adam(lr=0.001)</code>
Epoch	10–20
Early stopping	Stop if <code>val_loss</code> does not decrease after 3 epochs
Scheduler	<code>ReduceLROnPlateau</code> (optional)
Monitor	Train/val loss, save best model

Teacher Forcing

```
if random.random() < 0.5:
    decoder_input = target[:, t] # Ground truth
else:
    decoder_input = predicted_token # Prediction
```

6.4. Inference

- **Greedy decoding** : Choose the token with the highest probability.
- Stop when <eos> is encountered or **maximum length 50 is reached** .
- **Required** : Write a function `translate( sentence: str) -> str` receives an English sentence and returns a French sentence.

```
def translate(sentence):
    # Tokenize → tensor → encoder → decoder → detokenize
    return translated_french_sentence
```

6.5. Evaluation

Index	How to calculate
BLEU score	<code>nltk.translate.bleu_score.sentence_bleu</code> (on test set)
Perplexity	Options
Report	Average BLEU + 5 translation examples (correct/incorrect) + error analysis

7. Extensions (optional, + plus points)

1. Using **WMT 2014 dataset**.
2. Increase the number of LSTM layers or hidden size.
3. Replace **greedy decoding** with **beam search** (beam size = 3–5).
4. Add **attention mechanism** (Luong/Bahdanau).
5. Performance comparison with model with attention.

8. Tackle the difficult parts

Problem	Solution
Different chain lengths	<code>pack_padded_sequence</code> + sort batch by length
Teacher forcing & exposure bias	Use 0.5 ratio, or scheduled sampling
Overfitting	Dropout, sentence length limit (≤ 50), early stopping
Loss does not decrease	Test: learning rate, data normalization, shape tensor

9. Error analysis (required in report)

- Common errors:
 - Rare words (OOV) → `<unk>`
 - Long sentences → loss of information (due to fixed context vector)
 - Translation with incorrect grammar and missing words
- Suggested improvements:
 - Add attention
 - Use subword (BPE)

- Beam search

10. Rating scale (10 points)

Criteria	Point
1. Implement the correct model (Encoder-Decoder LSTM)	3.0
2. Data processing, DataLoader, padding/packing	2.0
3. Stable training, with early stopping, saving checkpoints	1.5
4. The <code>translate()</code> function works with new sentences	1.0
5. BLEU score evaluation + loss chart	1.0
6. Analysis of 5 error examples + suggestions for improvement	1.0
7. Source code quality (clean, commented, well structured)	0.5
8. Report (complete, clear, with charts, citations)	0.5
Plus points (expanded)	1.0

Total: 10

11. Important Note

1. **The source code must be executable from start to finish** on Google Colab or local machine.
 2. **The PDF report must include:**
 - Architectural diagram
 - Train/val loss chart
 - BLEU score
 - 5 translation examples + analysis
 - Source program (in Appendix)
 3. **Model checkpoint (`best_model.pth`) is required .**
 4. **Do not copy the code** → will get 0 points.
-

References

- Sutskever et al. (2014). *Sequence to Sequence Learning with Neural Networks* .
- PyTorch Documentation: [torch.nn.LSTM](#)
- Multi30K Dataset: <https://github.com/multi30k/dataset>